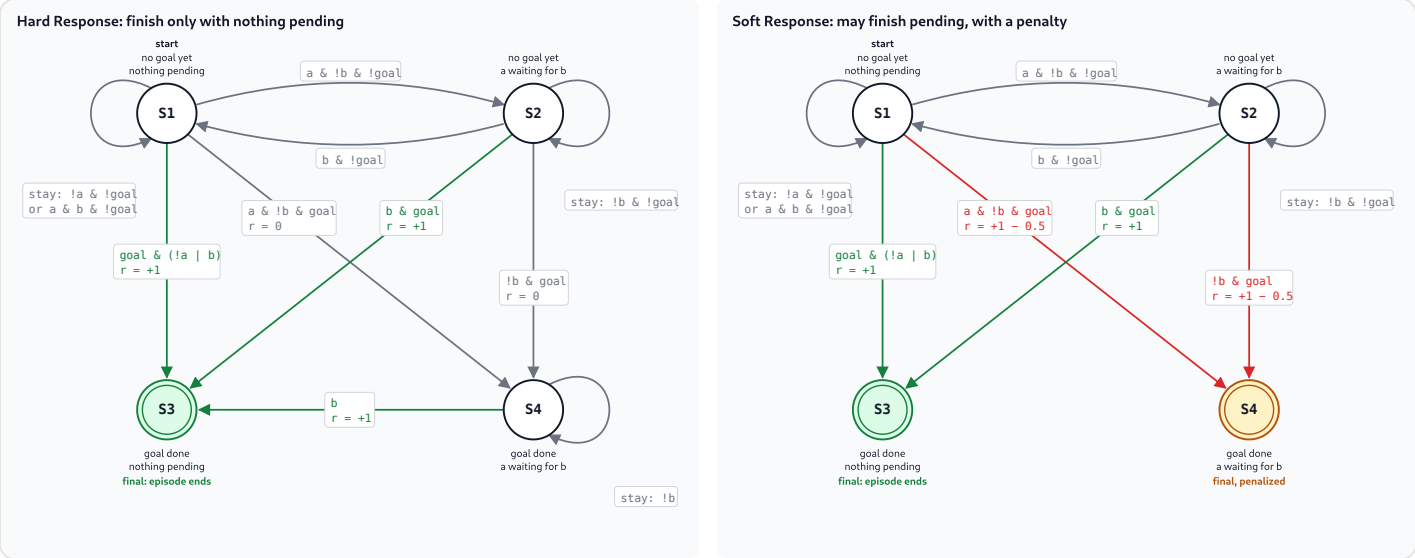
COMPILER · RULE CLAUSE + GOAL CLAUSE

Goal + Response: handled by the RM alone

Task: reach the goal (F goal), and every a must later be followed by b (G(a → F b)). Graphs come from FL-AT's real merged output for these two formulas.

Hard vs soft | Example walks | FL-AT output

Hard vs soft



Same four states in both. Hard: only S3 ends the episode, so reaching the goal with an a still waiting lands in S4 and the agent must still do b. Soft: S4 also ends the episode, but entering it pays +1 − 0.5. Open full size

- **No end-of-episode check needed.** Reaching a final state is what ends the episode, so the rule is judged on that last edge.
- **Hard = choose which states count as final.** FL-AT already marks only S3 as accepting.
- **Soft = also accept S4, and put the penalty on the edges into it.** 0.5 is an example value.
- **Only a Response-only task (no goal clause) has no final state.** The environment's time limit ends it.
- Simplified for reading: FL-AT's start S0 has exactly the same edges as S1, so it is drawn as S1. Progress shaping is left out; only the +1 task reward is shown.

Example walks

What the agent does	Hard	Soft
Reaches the goal with nothing pending	S1 → S3, +1, ends	S1 → S3, +1, ends
Does a, reaches the goal, then does b	S2 → S4 (0, keeps going) → S3 (+1), ends	S2 → S4, +0.5, ends at the

What the agent does	Hard	Soft
Does a and reaches the goal on the same step, no b	S4, 0, must still do b	S4, +0.5, ends
Does a, never b, never the goal	S2 until the time limit, 0	S2 until the time limit, 0

FL-AT output

▼ compile_reward_machine(["(Fgoal)", "(G(a->(Fb)))"], [0.1, 0.1])

```
accepting {'S3'} initial S0
  (S0, !a&!goal) -> S1  r=0.1
  (S0, !a&goal) -> S3  r=0.2
  (S0, a&!b&!goal) -> S2  r=0
  (S0, a&!b&goal) -> S4  r=0.1
  (S0, a&b&!goal) -> S1  r=0.1
  (S0, a&b&goal) -> S3  r=0.2
  (S1, !a&!goal) -> S1  r=0.1
  (S1, !a&goal) -> S3  r=0.2
  (S1, a&!b&!goal) -> S2  r=0
  (S1, a&!b&goal) -> S4  r=0.1
  (S1, a&b&!goal) -> S1  r=0.1
  (S1, a&b&goal) -> S3  r=0.2
  (S2, !b&!goal) -> S2  r=0
  (S2, !b&goal) -> S4  r=0.1
  (S2, b&!goal) -> S1  r=0.1
  (S2, b&goal) -> S3  r=0.2
  (S3, !a) -> S3  r=0.2
  (S3, a&!b) -> S4  r=0.1
  (S3, a&b) -> S3  r=0.2
  (S4, !b) -> S4  r=0.1
  (S4, b) -> S3  r=0.2
```

Regenerate: conda run -n RM_RL_env python -W ignore flat_response_goal.py from the repo root; graphs: python3 draw_response.py <this folder>.